技术部第四次技术分享会

基础数据结构与 Java 容器

姚雨森　技术部分管主席

回顾数组：长度需要提前确定

数组把一组同类型的数据放在一起，通过从 0 开始的下标访问。

```
int[] prices = new int[3];
prices[0] = 12;
prices[1] = 8;
prices[2] = 25;
System.out.println(prices.length); // 3
```

- 数组一旦创建，长度就固定了；可以修改元素，不能直接增加位置。
- 预留太少，后续数据放不下；预留太多，又有空余位置需要管理。
- 商品不断增加或下架时，我们希望能直接添加、删除元素，让保存的数据数量随需求变化。

除了位置编号，我们还需要数据的标签

数组用下标表示**位置**，但我们有时更想用一个**标签**找到数据。

- 按位置访问：第 0 件商品的价格是多少？
- 按标签查找：商品编号 P1001 对应的价格是多少？
- 这种用于查找的标签称为键（key），对应的数据称为值（value）。

数组也能配合查找代码完成任务，但我们希望有更直接的存储方式：

- **数量可变的列表**：方便添加、删除元素，不必自己安排数组长度。
- **键与值的映射**：根据商品编号等标签，保存和查询对应的数据。

ArrayList：可以增减元素的列表

可以把 ArrayList 理解为可以变化长度的数组，直接添加或删除元素。

```
// import java.util.ArrayList;
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
System.out.println(products.size()); // 2
System.out.println(products.get(0)); // Apple
products.set(0, "Bread");
products.remove(1);
System.out.println(products.size()); // 1
```

add 添加，get 读取，set 修改，remove 删除。

下标仍从 0 开始；元素数量用 size() 获取。

get、set、add：对应数组上的哪些操作

数组：用下标读取和赋值

```
String[] products = {"Apple"};

String first = products[0];

products[0] = "Bread";
```

ArrayList：调用对应的方法

```
ArrayList<String> products =
    new ArrayList<>();
products.add("Apple");
String first = products.get(0);
products.set(0, "Bread");
products.add("Milk");
```

- `get(i)` 对应数组的 `products[i]`：读取已有位置。
- `set(i, value)` 对应数组的下标赋值：替换元素，数量不变。
- `add(value)` 在末尾新增元素，数量加一；数组没有直接追加的操作。

空列表要先用 `add` 添加元素，不能用 `set(0, ...)` 创建第一个元素。

尖括号指定列表中的元素类型

ArrayList<String> 表示这个列表用来存放 String 类型的元素。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
String first = products.get(0);
// products.add(12); // Type mismatch
```

- 尖括号中的类型叫类型参数，这种写法称为泛型。
- 必须填写**引用类型**，例如 String、Student 等类。
- int、double 等**基本类型**不能直接填进去。

```
// ArrayList<int> prices = new ArrayList<>(); // Invalid
```

那么，要保存一组整数，应该用什么类型?

存放整数时，使用 Integer

保存 int 数据时，写成 ArrayList<Integer>；保存 double 数据时，使用 ArrayList<Double>。

```
ArrayList<Integer> prices = new ArrayList<>();
prices.add(12);
int price = prices.get(0);
```

基本类型	对应的类	基本类型	对应的类
byte	Byte	float	Float
short	Short	double	Double
int	Integer	char	Character
long	Long	boolean	Boolean

这些对应关系涉及**包装类**，这里先记住用法。